

FINAL PROJECT

Thermodynamical Approach to the Traveling Salesman Problem¹

통계학과 182STG15 이시영

DESCRIPTION

INTRODUCTION

- Monte Carlo Algorithm 을 사용하여 Traveling Salesman Problem 을 풀어본다.
- Traveling Salesman Problem 이란?

조합 최적화 문제의 일종으로 여러 도시들이 있고 한 도시에서 다른 도시로 이동하는 비용이 모두 주어졌을 때, 모든 도시들을 단 한번만 방문하고 원래 시작점으로 돌아오는 최소 비용의 이동 순서를 구하는 문제

- Monte Carlo 방법을 사용하여 이 문제의 근사적 해를 찾아본다.
- 통계적 열역학에서 주어진 Boltzmann-Gibbs 분포의 확률을 사용

BASIC PRINCIPLES

STATISTICAL THERMODYNAMICS

1 차원 상의 격자를 고려격자는 $\downarrow \uparrow$ 두가지 방향을 가지고 있다. N 개의 격자가 있을 때, 가능한 Configuration 의 수는 2^N 개이다.



각 Configuration 은 유한한 Energy 를 가진다고 가정한다. 에너지는 정의하기에 따라 다르다. 구조가 다르다고 에너지가 다를 필요는 없다.다음은 위의 Configuration 을 이용한 에너지 정의의 한 예이다.

EX. $E = B(n^+ - n^-)$, $n^+(n^-)$: # of arrows oriented up (down), B : Constant

EQUILIBRIUM STATE (평형 상태)

주어진 온도에서 시스템은 자발적으로 평형상태에 도달한다. 평형상태에 도달한다는 것은 에너지의 특정 평균값으로 다가간다는 것인데, 이 특정 평균값은 온도에 의해서 결정된다. 에너지의 평균값은 초기 configuration 과 관계 없다.

평형 상태는 정적인 상황이 아니다. Configuration 은 끊임없이 변화하는데, 시스템 특정 configuration 으로 변화할 확률은 Boltzmann-Gibbs distribution 에 의해서 주어진다.

$$P(conf) = C \exp(-E_{conf}/T)$$

E_{conf} : Configuration의 에너지, T : 온도, C : 정규화 상수

¹ V.Cerny. A Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm. 1985

평형시스템의 평균에너지는 다음과 같다.

$$\bar{E} = \left[\sum_{conf} E_{conf} \exp(-E_{conf}/T) \right] / \left[\sum_{conf} \exp(-E_{conf}/T) \right]$$

Configuration 의 수가 많을수록 $\bar{E}$ 의 계산이 어렵기에 Simulation 을 이용하여 $\bar{E}$ 를 계산한다.

이제 열역학 문제를 Traveling Salesman problem 과 연관짓자면, Traveling salesman trip 에서 Station 의 각 순열은 통계 시스템의 구성(Configuration)으로 간주된다. 시스템의 에너지는 여행의 총 길이와 같다. 온도는 임의의 실수이다. Monte Carlo 알고리즘을 사용하여 평형 상태에서 시스템의 구성을 다른 구성으로 무작위로 변경한다.

ALGORITHM

NOTATION

- Traveling salesman problem for N stations
- D: N by N matrix.

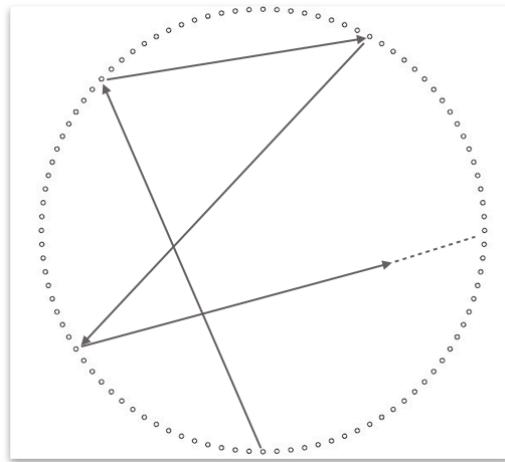
D(i,j): i 번째 station 과 j 번째 station 사이의 거리.

- 정수 1,2,...,N 로 이루어진 순열

$\{s_i\}$: Starting, $\{c_i\}$: Current, $\{t_i\}$: Trial, $i = 1,2,...,N$

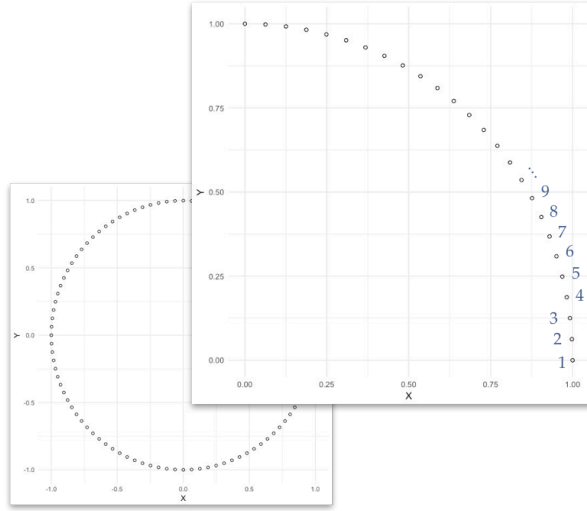
- 총 거리: $d = D(c_N, c_1) + \sum_{k=1}^{N-1} D(c_k, c_{k+1})$
- 임의의 실수 T: 0.1, 0.01, 0.001

EXAMPLE 1 WITH ALGORITHM



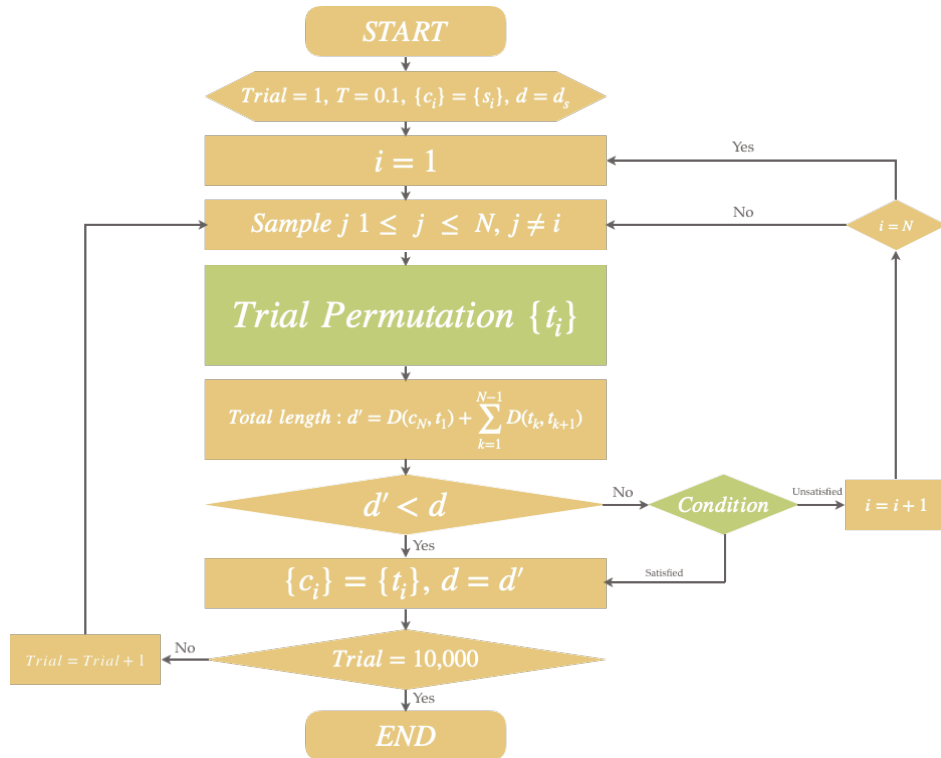
100 개의 점이 Uniform 하게 분포된 단위원을 고려한다. 점들 사이의 거리는 유클리드 거리를 이용한다. 모든 점을 다 거치고 원래 시작점으로 돌아오는 최소 거리의 이동 순서를 구하여라.

(1,0)에 있는 점을 1 번 점으로 설정한 후 반 시계 방향으로 번호를 설정한다.



■ Algorithm

임의의 순열 한 개를 뽑고 Starting 순열 $\{s_i\}$ 로 지정한다. $\{s_i\} = \{3, 49, 8, \dots, 2, 54, 76\}$, Total length $d_s = 133.5281$, $T=0.1, 0.01, 0.001$ 에 대하여 다음의 알고리즘을 시행한다.



• Trial Permutation

$\tilde{i} = \min(i, j), \tilde{j} = \max(i, j), t_k = c_k, k = 1, 2, \dots, i - 1$

$t_{\tilde{i}+k} = c_{\tilde{j}-k}, k = 0, 1, \dots, \tilde{j} - \tilde{i}$

$t_k = c_k, k = \tilde{j} + 1, \tilde{j} + 2, \dots, N$

Ex. $\tilde{i} = 3, \tilde{j} = 7$

$\{c_i\} :$ 1 2 3 4 5 6 7 8 9 10

$\{t_i\} :$ 1 2 7 6 5 4 3 8 9 10

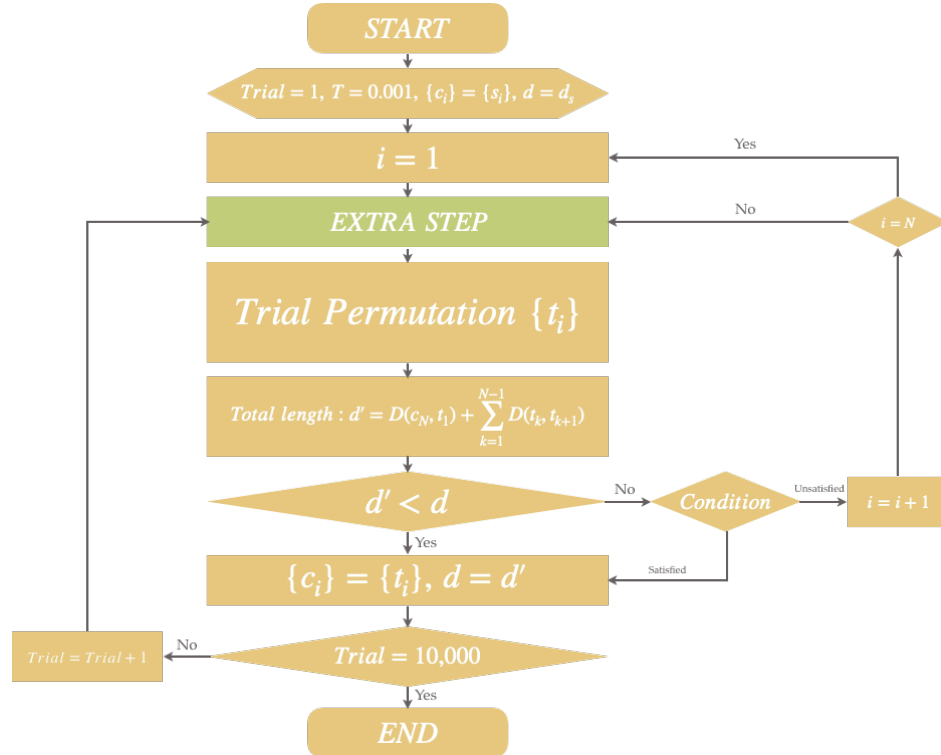
기존 순열을 고정하고 선택된 $\tilde{i}, \tilde{j}$ 를 이용하여 순서를 다음과 같이 역행으로 바꾼다.

- **Condition**

Generate $x \sim \text{Unif}(0,1)$, $x < \exp[(d - d')/T]$

Boltzmann Factor: $P(\text{conf}) = C \exp(-E_{\text{conf}}/T)$

Trial 순열은 Boltzmann Factor 에 의해 주어진 확률로 현재 순열로 받아들여진다.



- **Extra Step**

시스템이 평형 상태에 도달했을 때, 순열의 변화를 좀더 국부적으로 변화시키기 위하여 다음 단계를 추가한다.
기존의 j 대신 m 을 사용한다.

Sample $j, 1 \leq j \leq N, j \neq i$

$m = \min\{|j - i|, |j - i + N|, |j - i - N|\}$

$1 \leq m \leq n < \frac{N}{2}, n: \text{Some fixed number} = N/10$

EXAMPLE 2

Example1 에서 이어진다. $\tilde{D}$ = Distance Matrix from Example 1 라 할 때, Distance Matrix 를 다음과 같이 정의한다.

$$D(i, j) = \tilde{D}(i, j) + P_i + P_j, i, j = 1, 2, \dots, 100$$

$$P_i = i\sqrt{2} - \lfloor i\sqrt{2} \rfloor$$

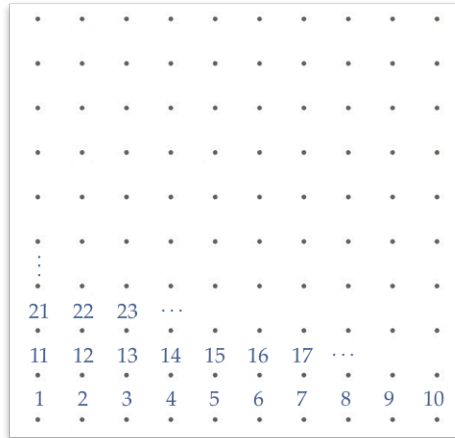
- **Distance Matrix D:**

	1	2	3	...	100
1	0.828	1.305	0.782		0.898
2	1.305	1.657	1.134		1.375
3	0.782	1.134	0.485		0.852
⋮				⋱	
100	0.898	1.375	0.852		0.843

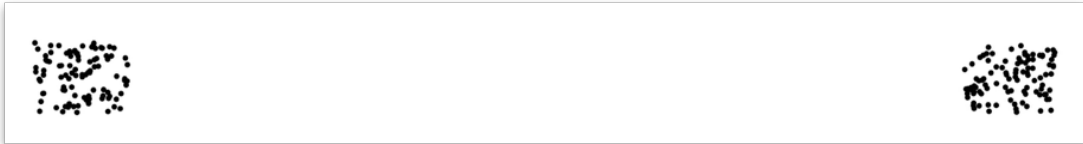
위의 Distance Matrix 를 이용하여 최소 거리의 이동 순서를 구하여라.

EXAMPLE 3

다음과 같이 거리 1 을 둔 100 개의 점 고려할 때, 최소 거리의 이동 순서를 구하여라.



EXAMPLE 4

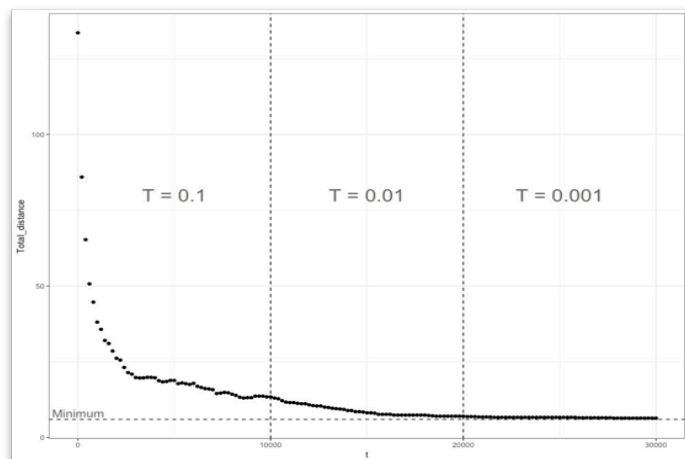
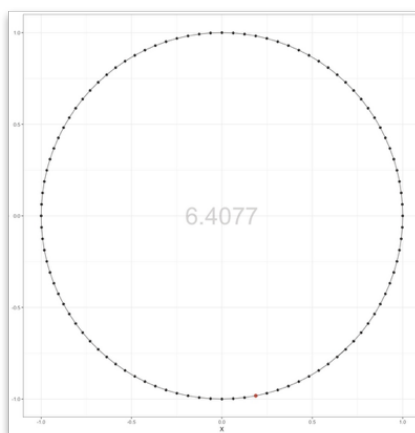


200 개의 점을 생성. 100 개의 점을 길이 1 인 정사각형 안에 uniform 하게 생성한다. 처음 정사각형과 거리 10 을 두고 두번째 길이 1 인 정사각형 안에 100 개의 점을 uniform 하게 생성한다. 이 때 최소 거리의 이동 순서를 구하여라. 이 문제를 통해 알고리즘이 얼마나 빠르게 불필요한 경로를 제거하는지 확인할 수 있다.

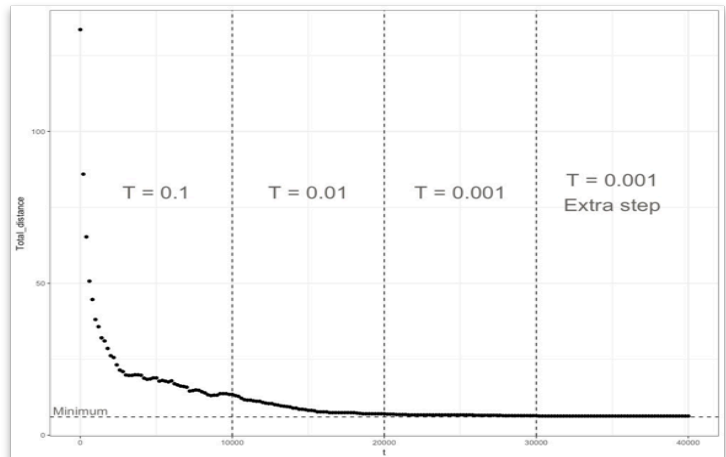
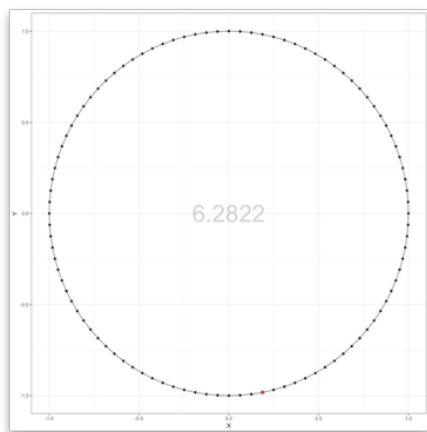
IMPLEMENTATION

EXAMPLE 1

$T=0.1, 0.01, 0.001$ 일때 10,000 번씩 시뮬레이션을 진행한 결과는 다음과 같다.



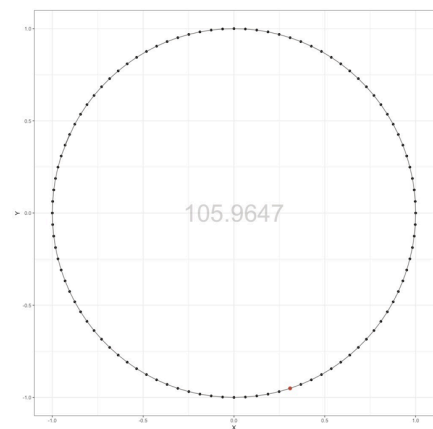
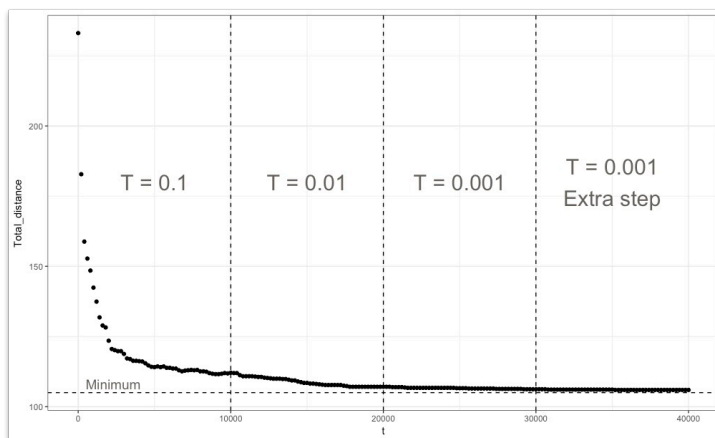
그 후 Extra Step 을 진행한 결과



가장 Optimal 한 값을 찾은 것을 알 수 있다.

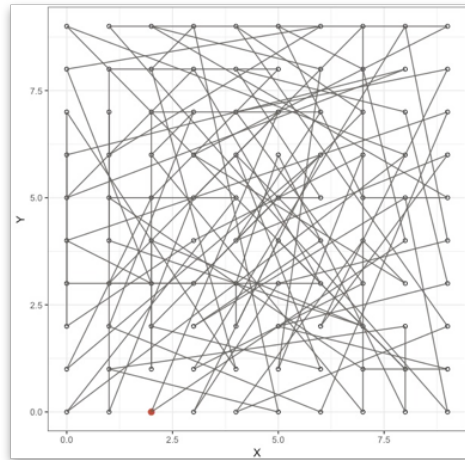
EXAMPLE 2

바뀐 Distance matrix 를 사용하여 알고리즘을 시행하였을 때 결과는 다음과 같다. Scale 은 커졌지만 1 번 예제와 비슷한 양상을 띈다. Optimal Solution(순열)은 바뀌지 않았다. 하지만 기하학적으로는 완전히 다른 문제가 되었다.

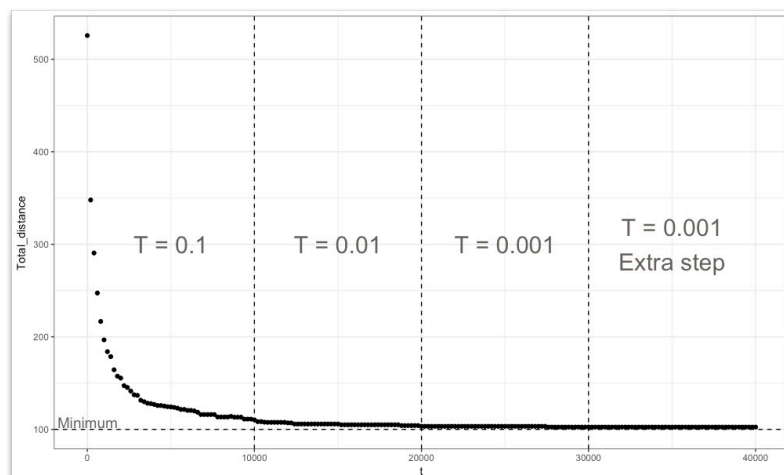
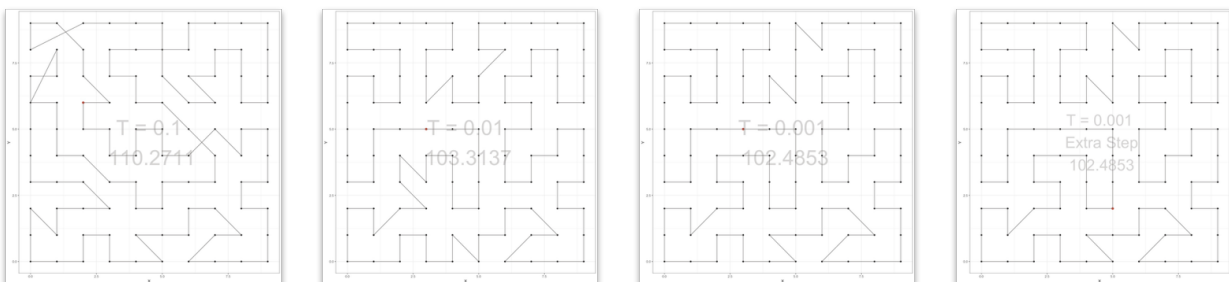


EXAMPLE 3

임의의 Starting 순열을 뽑아 경로를 그려보면 다음과 같다.



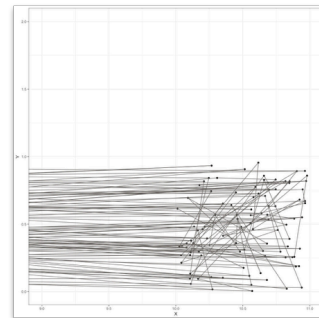
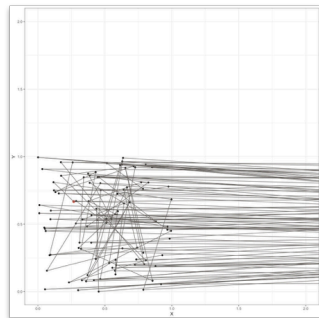
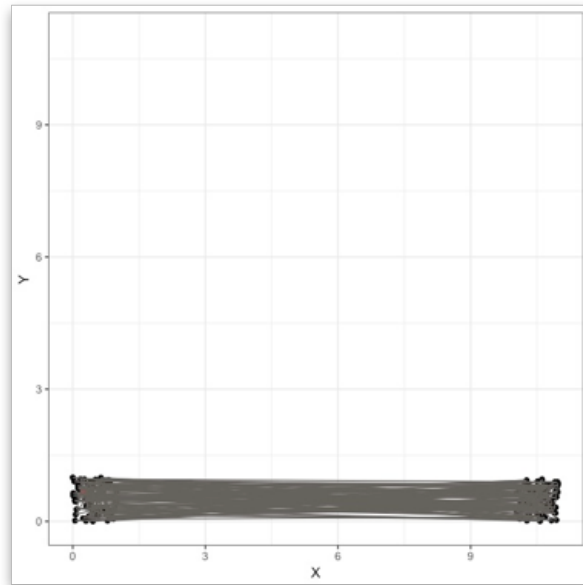
$T=0.1, 0.01, 0.001$ 에 따라 시뮬레이션을 진행한 결과는 다음과 같다.



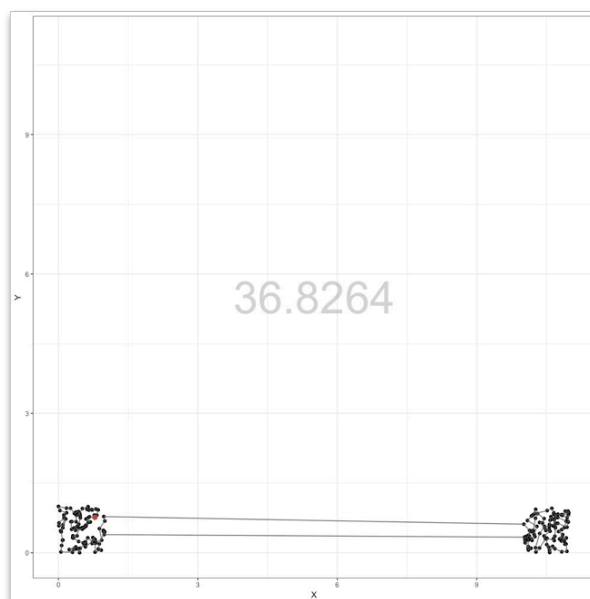
완벽한 Optimum 으로 도달하지는 못하였다.

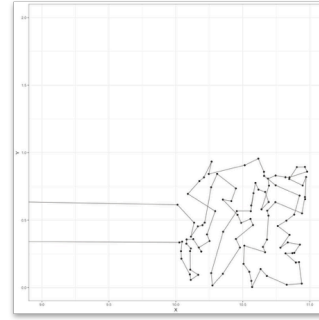
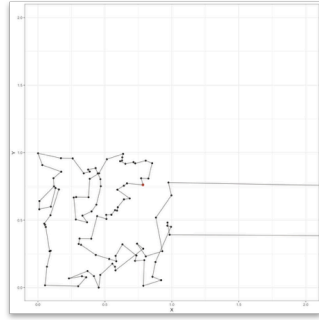
EXAMPLE 4

임의의 뽑은 Starting 순열로 그린 경로는 다음과 같다.

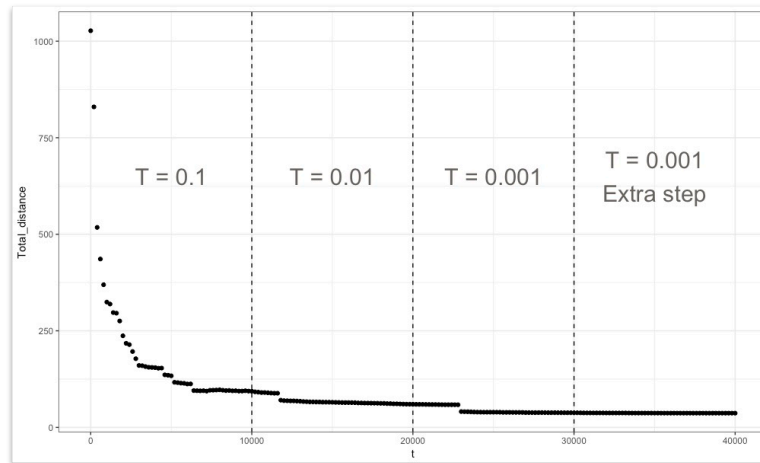


$T=0.1, 0.01, 0.001$ 에 따라 시뮬레이션을 진행한 결과는 다음과 같다.





198 번의 불필요한 이동을 제거하였다.



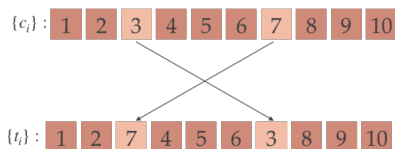
DISCUSSION

Algorithm 과 관련하여 이론적으로 논하기에는 어려움이 있으나 실용적으로 활용하기에 적절하다. 또한 범용적 최적화 절차로서 가능성이 보인다. 이 Algorithm 은 Configuration ξ 와 최소화 할 함수 $F(\xi)$ 두가지 개념(concept)으로 작동한다. 이 두가지 개념은 최적화 문제에서 전형적으로 많이 사용되는 개념이다. Current Configuration ξ 와 비슷한 Trial Configuration $\hat{\xi}$ 을 생성한다. $F(\hat{\xi}) < F(\xi)$ 의 경우 $\hat{\xi}$ 을 채택하며 $F(\hat{\xi}) > F(\xi)$ 경우의 Acceptance 를 논문은 Boltzmann Factor 를 이용해 결정하였다.

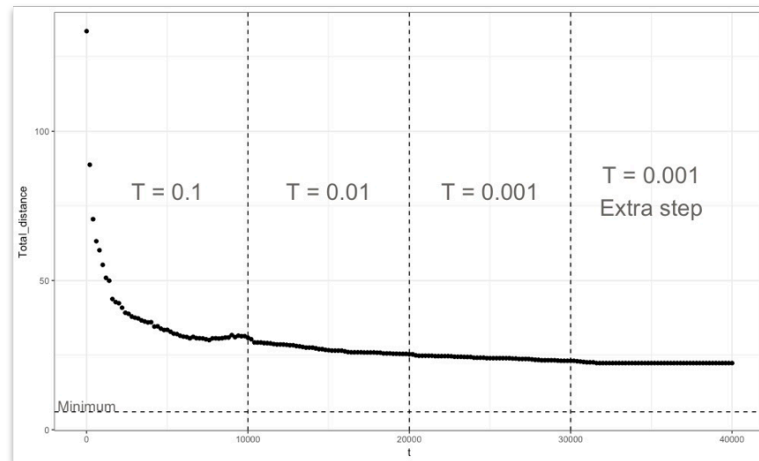
“Current Configuration ξ 와 비슷한 Trial Configuration $\hat{\xi}$ 을 생성한다.”에서 어느정도 비슷한 configuration 을 생성할 지에 대한 고려가 필요한데 만약 기존 configuration 과 너무 비슷한 경우 local optimum 에 빠질 가능성이 있다. 이 사실은 다음 예제를 통해 알 수 있다.

- Trial Permutation
 $t_k = c_k, k = 1, 2, 3, \dots, N, k \neq i, j$
 $t_i = c_j, \quad t_j = c_i$

Ex. $i = 3, j = 7$



해당 Permutation 방법을 사용하여 Example1 에 Algorithm 을 수행하면 다음과 같은 결과가 나온다.



질적인 양상은 비슷하나 양적으로 매우 다르다. 6 과 비슷한 값이 나와야 best optimum 이지만 20 근처에서 멈춰 있기에 Local optimum 에 빠진 것을 알 수 있다. 그렇다고 기존의 방법이 Universal 한 방법이라는 것은 아니며 경험상 방법을 바꿔가며 문제에 적절한 방법을 사용해야 한다.

온도를 사용하여 Local minimum 에서 빠져 나올 수도 있다. 온도를 증가하면 변동이 커지기에 Local minimum 을 빠져나올 가능성을 부여한다. 열역학에서 평형 상태는 특정 제약 조건 하에 최대 엔트로피를 갖는 상태이다. 이러한 의미에서 평형상태로의 전환은 최적화 문제이다.

APPENDIX

사용 Library

```
library(ggplot2)
```

Calculate Total Length

```
total_dist <- function(pmt, D, N){  
  
  dist <- D[pmt[N],pmt[1]]  
  
  for(k in 1:(N-1)){  
  
    dist <- dist + D[pmt[k],pmt[k+1]]  
  
  }  
  
  return(dist)  
}
```

Trial Permutaion

```
trial_pmt <- function(curr_pmt, i, j, N){  
  
  ii <- min(c(i, j)); jj <- max(c(i, j)); t_pmt <- NA  
  
  if(ii != 1){  
  
    t_pmt[1:(ii-1)] <- curr_pmt[1:(ii-1)]  
  
  }  
  
  for(k in (0:(jj-ii))){  
  
    t_pmt[ii + k] <- curr_pmt[jj - k]  
  
  }  
  
  if(jj != N){  
  
    t_pmt[(jj+1):N] <- curr_pmt[(jj+1):N]  
  
  }  
  
  return(t_pmt)  
}
```

Trial Permutaion 2nd Option

```
trial_pmt2 <- function(curr_pmt, i, j, N){  
  
  t_pmt <- curr_pmt  
  
  t_pmt[i] <- curr_pmt[j]  
  
  t_pmt[j] <- curr_pmt[i]  
  
  return(t_pmt)  
}
```

Algorithm

```
algo <- function(N, D_matrix, start_pmt, trial_method = trial_pmt, extra_step = F, trace = F, ntrial = 10^4){
```

```

result <- list()

i = 1; curr_pmt <- start_pmt; curr_d <- total_dist(curr_pmt, D_matrix, N)

storage <- data.frame(t = 0, Total_distance = curr_d); trial <- 0

storage_pmt <- matrix(NA, ncol = N, nrow = 4); xx <- 1

for(Temp in c(0.1, 0.01, 0.001)){

  for(t in 1:ntrial){

    j <- sample(seq(1,N)[-i], 1)

    t_pmt <- trial_method(curr_pmt, i, j, N); dprime <- total_dist(t_pmt, D_matrix, N)

    if(dprime < curr_d){

      curr_pmt <- t_pmt; curr_d <- total_dist(curr_pmt, D_matrix, N)

    }else{

      x <- runif(1)

      if(x < exp((curr_d-dprime)/Temp)){

        curr_pmt <- t_pmt; curr_d <- total_dist(curr_pmt, D_matrix, N)

      }else{

        i <- ifelse(i < N, i+1, 1)

      }

    }

    trial <- trial + 1

    #Trace

    if((t %% 200 == 0) & (trace == T)){

      if((t == ntrial)){storage_pmt[xx,] <- curr_pmt; xx <- xx + 1}

      storage <- rbind(storage, data.frame(t = trial, Total_distance = curr_d))

      print(paste("TEMP:", Temp, "Trial:", t, "Total Distance:", round(curr_d,4)))

    }

  }

}

#Extra Step

if(extra_step == T){

  Temp <- 0.001; i <- 1; n <- N/10

  for(t in 1:ntrial){

    m <- n + 1

    while((m >= (n+1)) | (m == 0)){

```

```

j <- sample(seq(1,N)[-i], 1)

m <- min(c(abs(j-1), abs(j-i+N), abs(j-i-N)))
}

t_pmt <- trial_method(curr_pmt, i, m, N); dprime <- total_dist(t_pmt, D_matrix, N)

if(dprime < curr_d){
  curr_pmt <- t_pmt; curr_d <- total_dist(curr_pmt, D_matrix, N)
}else{
  x <- runif(1)

  if(x < exp((curr_d-dprime)/Temp)){
    curr_pmt <- t_pmt; curr_d <- total_dist(curr_pmt, D_matrix, N)
  }else{
    i <- ifelse(i < N, i+1, 1)
  }
}

trial <- trial + 1

#Trace
if((t %% 200 == 0) & (trace == T)){
  if((t == ntrial)){storage_pmt[xx,] <- curr_pmt}

  storage <- rbind(storage, data.frame(t = trial, Total_distance = curr_d))

  print(paste('Extra :: TEMP:', Temp, "Trial:", t, 'Total Distance:', round(curr_d,4)))
}
}

#Result
if(trace == T){
  result$Permutation <- curr_pmt
  result$Total_distance <- curr_d
  result$Storage <- storage
  result$Storage_pmt <- storage_pmt
} else{
  result$Permutation <- curr_pmt
  result$Total_distance <- curr_d
}

```

```

return(result)

}

```

Graph

```

route_graph <- function(pmt, df, N, lim=NULL){

  p <- ggplot(data = df, aes(x = X, y = Y)) + geom_point() +

    geom_point(data = df[pmt[1],],aes(x=X, y=Y), color = '#C94A36', size = 2.5)

  for(i in 1:(N-1)){

    p <- p + geom_line(data = df[c(pmt[i], pmt[i+1]),], aes(x = X, y = Y), color = '#66635F')

  }

  p <- p + geom_line(data = df[c(pmt[N], pmt[1]),], aes(x = X, y = Y), color = '#66635F') + theme_bw()

  if(!is.null(lim)){

    p <- p + coord_cartesian(xlim = c(lim$x[1], lim$x[2]), ylim = c(lim$y[1], lim$y[2]))

  }

  return(p)

}

```

EXAMPLE 1

```

set.seed(109)

###INITIAL SETTING###

theta <- seq(from= 0, to= 2*pi-2*pi/100, length = 100)

x <- cos(theta); y <- sin(theta)

dat1 <- data.frame(X = x, Y = y)

ggplot(dat1, aes(x= X, y = Y)) + geom_point(shape =1) + theme_minimal() +

  coord_cartesian(xlim = c(-1,1), ylim = c(-1,1)) +

  theme(line = element_blank(),text = element_blank(),title = element_blank())


#DISTANCE MATRIX

#D: 100 BY 100 MATRIX

D <- as.matrix(dist(dat1))

s <- sample(seq(1:100), 100)

total_dist(s,D,100)

route_graph(s, dat1, 100) + theme_minimal()


result1 <- algo(100, D, s, extra_step = T, trace = T, ntrial = 2*10^4)

result1$Storage

```

```

route_graph(result1$Permutation, dat1, 100)

ggplot(result1$Storage, aes(x = t, y = Total_distance)) +

  geom_point() + theme_bw() +

  geom_vline(xintercept = 20000, linetype='dashed') + geom_vline(xintercept = 40000, linetype='dashed') +

  geom_hline(yintercept = 6, linetype = 'dashed') + geom_vline(xintercept = 60000, linetype='dashed') +

  annotate('text', x = 10000, y = 80, label = 'T = 0.1', size = 8, color = '#66635F') +

  annotate('text', x = 30000, y = 80, label = 'T = 0.01', size = 8, color = '#66635F') +

  annotate('text', x = 50000, y = 80, label = 'T = 0.001', size = 8, color = '#66635F') +

  annotate('text', x = 500, y = 10, label = 'Minimum', size = 5, color = '#66635F') +

  annotate('text', x = 70000, y = 80, label = 'T = 0.001\nExtra step', size = 8, color = '#66635F')

route_graph(result1$Storage_pmt[1,], dat1, 100)

route_graph(result1$Storage_pmt[2,], dat1, 100)

route_graph(result1$Storage_pmt[3,], dat1, 100)

route_graph(result1$Storage_pmt[4,], dat1, 100)

```

#j Sampling -> m Sampling

```

result1.1 <- algo(100, D, s, trial_method = trial_pmt2, extra_step = T, trace = T, ntrial = 2*10^4)

ggplot(result1.1$Storage, aes(x = t, y = Total_distance)) +

  geom_point() + theme_bw() +

  geom_vline(xintercept = 20000, linetype='dashed') + geom_vline(xintercept = 40000, linetype='dashed') +

  geom_hline(yintercept = 6, linetype = 'dashed') + geom_vline(xintercept = 60000, linetype='dashed') +

  annotate('text', x = 10000, y = 80, label = 'T = 0.1', size = 8, color = '#66635F') +

  annotate('text', x = 30000, y = 80, label = 'T = 0.01', size = 8, color = '#66635F') +

  annotate('text', x = 50000, y = 80, label = 'T = 0.001', size = 8, color = '#66635F') +

  annotate('text', x = 500, y = 10, label = 'Minimum', size = 5, color = '#66635F') +

  annotate('text', x = 70000, y = 80, label = 'T = 0.001\nExtra step', size = 8, color = '#66635F')

```

EXAMPLE 2

```

set.seed(110)

###INITIAL SETTING###

D.2 <- matrix(NA, ncol = 100, nrow = 100)

i <- seq(1,100)

P <- i*sqrt(2) - floor(i*sqrt(2))

for(j in 1:100){

```

```

for(i in 1:100){
  D.2[i, j] = D[i,j] + P[i] + P[j]
}
}

result2 <- algo(100, D.2, s, extra_step = T, trace = T)

total_dist(result2$Permutation,D.2,100)

route_graph(result2$Permutation, dat1, 100) +

  annotate('text', x = 0, y = 0, label = '105.9647', size = 15, color = '#66635F', alpha = 0.3)

ggplot(result2$Storage, aes(x = t, y = Total_distance)) + geom_point() + theme_bw() +

  geom_vline(xintercept = 10000, linetype='dashed') + geom_vline(xintercept = 20000, linetype='dashed') +

  geom_vline(xintercept = 30000, linetype='dashed') + geom_hline(yintercept = 105, linetype = 'dashed') +

  annotate('text', x = 5000, y = 180, label = 'T = 0.1', size = 8, color = '#66635F') +

  annotate('text', x = 15000, y = 180, label = 'T = 0.01', size = 8, color = '#66635F') +

  annotate('text', x = 25000, y = 180, label = 'T = 0.001', size = 8, color = '#66635F') +

  annotate('text', x = 35000, y = 180, label = 'T = 0.001\nExtra step', size = 8, color = '#66635F')+

  annotate('text', x = 2300, y = 108, label = 'Minimum', size = 5, color = '#66635F')

```

EXAMPLE 3

```

X.3 <- rep(seq(0,9),10); Y.3 <- as.vector(sapply(seq(0,9), function(x) rep(x, 10)))

dat3 <- data.frame(X=X.3, Y=Y.3); D.3 <- as.matrix(dist(dat3))

ggplot(dat3, aes(x= X, y= Y)) + geom_point(color = '#66635F') + theme_minimal() +

  coord_cartesian(xlim = c(0,9), ylim = c(0,9)) + theme(line = element_blank(),text = element_blank(), title = element_blank())

total_dist(s ,D.3, 100)

result3 <- algo(100, D.3, s, extra_step = T,trace = T)

route_graph(result3$Permutation, dat3, 100)

ggplot(result3$Storage, aes(x= t, y= Total_distance)) + geom_point() + theme_bw() +

  geom_vline(xintercept = 10000, linetype='dashed') + geom_vline(xintercept = 20000, linetype='dashed') +

  geom_vline(xintercept = 30000, linetype='dashed') + geom_hline(yintercept = 100, linetype = 'dashed') +

  annotate('text', x = 5000, y = 300, label = 'T = 0.1', size = 8, color = '#66635F') +

  annotate('text', x = 15000, y = 300, label = 'T = 0.01', size = 8, color = '#66635F') +

  annotate('text', x = 25000, y = 300, label = 'T = 0.001', size = 8, color = '#66635F') +

  annotate('text', x = 35000, y = 300, label = 'T = 0.001\nExtra step', size = 8, color = '#66635F') +

  annotate('text', x = 18, y = 108, label = 'Minimum', size = 5, color = '#66635F')

```

```

route_graph(result3$Storage_pmt[1,], dat3, 100)

total_dist(result3$Storage_pmt[1,], D.3, 100)

route_graph(result3$Storage_pmt[2,], dat3, 100)

total_dist(result3$Storage_pmt[2,], D.3, 100)

route_graph(result3$Storage_pmt[3,], dat3, 100)

total_dist(result3$Storage_pmt[3,], D.3, 100)

```

EXAMPLE 4

```

set.seed(111)

X.4 <- c(runif(100), runif(100, min=10, max=11)); Y.4 <- runif(200)

dat4 <- data.frame(X=X.4, Y=Y.4)

ggplot(dat4, aes(x= X, y = Y)) + geom_point() +

  coord_cartesian(xlim = c(0,11), ylim = c(0,11)) + theme_classic() +

  theme(line = element_blank(), text = element_blank(), title = element_blank())


s2 <- sample(seq(1,200),200)

D.4 <- as.matrix(dist(dat4))

route_graph(s2, dat4, 200, lim = list(x=c(0,11), y=c(0,11)))

route_graph(s2, dat4, 200, lim = list(x=c(0,2), y=c(0,2)))

route_graph(s2, dat4, 200, lim = list(x=c(9,11), y=c(0,2)))


result4 <- algo(200, D.4, s2, extra_step = T, trace = T)

route_graph(result4$Permutation, dat4, 200, lim = list(x=c(0,11), y=c(0,11)))

route_graph(result4$Permutation, dat4, 200, lim = list(x=c(0,2), y=c(0,2)))

route_graph(result4$Permutation, dat4, 200, lim = list(x=c(9,11), y=c(0,2)))


ggplot(result4$Storage, aes(x= t, y= Total_distance)) + geom_point() + theme_bw() +

  geom_vline(xintercept = 10000, linetype='dashed') + geom_vline(xintercept = 20000, linetype='dashed') +

  geom_vline(xintercept = 30000, linetype='dashed') +

  annotate('text', x = 5000, y = 650, label = 'T = 0.1', size = 8, color = '#66635F') +

  annotate('text', x = 15000, y = 650, label = 'T = 0.01', size = 8, color = '#66635F') +

  annotate('text', x = 25000, y = 650, label = 'T = 0.001', size = 8, color = '#66635F') +

  annotate('text', x = 35000, y = 650, label = 'T = 0.001\nExtra step', size = 8, color = '#66635F')

```